

Università
della
Svizzera
italiana

Faculty
of
Informatics

Bachelor Thesis

June 23, 2023

Cryptocurrency Market Modelling Using Causal Inference Methods and Sentiment Analysis

How to analyse the the relationship between sentiments and cryptocurrencies

Lovnesh Bhardwaj

Abstract

Financial Market Modelling is an age-old problem that fascinates economists and mathematicians alike. Market models allow traders to look at how the financial instruments in any given market interact to predict positive returns. Most market modelling uses general statistical frameworks that rely heavily on correlations between the instruments. Correlations can be spurious and, therefore, are not the best indicator for the relationships between different financial instruments, as, during uncertain periods, a model based on correlations would perform poorly. We use causal inference, a specialised statistical framework, to monitor causal dependencies between instruments in the cryptocurrency market and their sentiments to provide a more robust market model. It was found that Twitter sentiments have no causal effect on the returns of the cryptocurrencies and, therefore, can be ignored while modelling the market and that the market impacts itself.

Advisor
Prof. Ernst-Jan Camiel Wit
Assistant
Ms. Katerina Rigana

Advisor's approval (Prof. Ernst-Jan Camiel Wit):



Date: 23 June 2023

Contents

1 Introduction **2**

2 Background to Causal Contagion Model **3**

2.1 Causal Inference 3

2.2 Graphs, DAGs, Graphical Models, and do-calculus 4

2.3 From Directed Acyclic Graph to Structural Causal Model 5

2.4 The PC Algorithm 7

2.4.1 Inferring the Undirected Graph 7

2.4.2 Orientation of the edges to form Causal Links 8

2.5 Normality and Copulas 9

3 Introduction to the Network Description of Contagion **10**

4 Data Collection **11**

4.1 Twitter Data Collection 11

4.1.1 Fetching Tweets Using API 11

4.1.2 Twitter (Nitter) Scraping Using Scrapy 13

4.2 Binance Price Fetching 14

5 Exploratory Analysis and Summary Statistics **15**

5.1 Pre-processing and Sentiment Analysis 15

5.2 Data Analysis for Prices 18

6 Application of Causal Contagion to Cryptocurrencies and their Sentiments **20**

7 Conclusion and Discussion **22**

1 Introduction

The problem of predicting the market is an ancient one. Some traders may predict an asset's price or return over time by analysing the data at hand, and others simply go with their gut. However, most professionals use data analysis to provide them with the information to potentially generate positive returns, with many focusing on Technical Analysis of the data [12] to create market models. Technical analysis, as we know it now, was introduced by Dow in the late 1800s [16]. Many later built on top of the work by Dow, most notably the work of John Magee, who wrote the book "Technical Analysis of Stock Trends", which is regarded by many as being the bible of Technical Analysis. However, he was known to go against his charts and trade based on his gut feelings, or in simpler terms, sentiments [1].

Sentiments play a crucial role in the financial markets. It has been shown from previous works that news impacts financial markets and that investor sentiments are affected by said news [37]. Pearson further states that decision-making in financial markets is driven by psychological factors and emotions and that, therefore, market behaviour is not always synonymous with the fundamental value of the asset [37][22]. Sentiments in the context of this paper will therefore be defined as discrepancies between the current value of an asset from its actual value (which is subjective for the investor); therefore, if an investor thinks that the current price of an asset is below its true value, they will have a positive outlook towards the future price of the said asset, whereas if the current price of an asset is above its true value, the investor will have a negative outlook towards the future price of the asset.

This act of predicting the value of an asset (with or without using sentiments), or "Timing the Market", goes against the Efficient Market Hypothesis (EMH), which states that if a market is efficient, then the price changes over time of an asset are independent, or that the market prices reflect all available market information [13]. This means that you cannot predict the prices of an asset in the market. However, there is evidence that financial markets are not efficient. Studies show that there is strong evidence of co-movement of investor sentiments with the market but little evidence of short-run predictability in returns [4]. There is also positive evidence of a correlation between Twitter sentiments and the stock market [2]. Another assumption by EMH states that an investor in the market is rational and diversifies to optimise the statistical value of their portfolio and that the value of an asset in the market at any given time is its true value (classical finance theory). But, the co-existence of irrationality and rationality is the truth in any given market, and therefore, this assumption of EMH does not fully encapsulate the market landscape. To mitigate that, we will use the Adaptive Market Hypothesis (AMH), which takes into account the short-comings of EMH, and argues that, while it is not wrong, it is just incomplete as it fails to take into account the irrationality of the market [25], that is, taking the Investor Sentiment into account, saying that the markets are "varying in their degrees of efficiencies as the environment and investor population changes over time" [26].

There are many ways in which sentiments are measured in different studies, and several databanks are available, like the Sentix Survey (see www.sentix.de) or CBOE Volatility Index (VIX), which quantifies human sentiments for a given asset. Other surveys include the American Association of Individual Investors (AAII) or Investor Intelligence (II). In this study, however, I will use Twitter (see www.twitter.com) as the source of sentiments of the population towards the price of an asset. Twitter is better than traditional methods of surveying for several reasons, the most notable being that one can live stream events on Twitter, which means we can have an idea of the general perception of the pricing of an asset in the market live, meaning that the tweets are much faster than other forms of sentiment retrieval like surveys (like the ones mentioned above), news articles, amongst other things. Some studies have used tweet sentiments to predict asset price using bivariate granger-causality tests [22], others have chosen a more straightforward route of standard machine learning techniques to form cryptocurrency trading models using Twitter sentiment analysis [8]. The literature pertaining to the model on how the instruments of the Cryptocurrency market interact is very limited. We would therefore be expanding upon that area and incorporate Investor Sentiment, by the way of tweets, to model the Cryptocurrency Market, specifically 8 different cryptocurrencies, namely *Bitcoin, Polygon (Matic), Ripple, Polkadot, Dogecoin, Cardano, Hedera, and Solana*.

Cryptocurrencies are digital currencies which reside on a blockchain, and they were introduced in the paper by a person going by the pseudonym Nakamoto in the year 2008 [28]. They were introduced as a way to not rely on third-party merchants to process transactions between two people. That is, it provides an alternative to the trust-based model in which two transacting parties must rely on a third party (often a bank) to process a transaction between party A and party B. Since the inception of the cryptocurrencies in the form of bitcoin, the market has expanded rapidly with the total market capitalisation being $1.1 trillion, and 22,932 cryptocurrencies being in the market as of now [17]. Their real potential, and the fact that the cryptocurrency market is relatively new and unregulated, has led to there being an explosive growth of interest in them, most notably in the years 2017-2018 due to reports of massive returns and 2020-2021 "due to the macroeconomic instability resulting from unchecked stimulus during the economic shutdown" [42]. This is consistent with the fact that cryptocurrencies can be used as a hedging tool in times of global uncertainity [3][24] even though they are not a good example of one.

It is challenging to determine what type of asset cryptocurrencies are. They are not currencies in the traditional sense, given that they lack a guarantor or that their value remains uncertain over time. The cryptocurrency markets,

on the other hand, are incredibly volatile, and thus make them unsuitable as a reliable storage of value or a medium of exchange [7]. In 2021, the value of assets was increasing between 50%-100% (MATIC in mid-2021), while the others, such as Dogecoin and Shiba Inu, skyrocketed gradually with more than 1000% increases within a year, which is a testament to the volatility of the cryptocurrency market. Due to this volatility, and in the case of some scarce cryptocurrencies, the cryptocurrency market cannot be deemed a traditional currency market, and it can also be said that cryptocurrencies are not "real" currencies [50].

It can be said that the cryptocurrency market is very interdependent [6], which we will use to model the cryptocurrency market using various cryptocurrency tokens (referred to as crypto tokens or tokens from now on) and sentiments. As stated earlier, some studies further show there has already been a significant bidirectional relationship between bitcoin prices and sentiment [23]. Many studies rely on a basic statistical framework to establish a correlation between prices and sentiments. In this paper, I will employ a causal framework (a special type of statistical framework) whereby we will see if there are causal links between cryptocurrency prices and sentiments and within prices and sentiments, respectively, for modelling the market. That is, if one price of one asset causes the other to change, if the sentiment of one asset causes the price of that asset to change, or if the price of the asset causes the sentiment of that asset to change.

The rest of this report is structured as follows: Section 2 will provide the background for the causal framework used to analyse the data collected. Section 3 would provide an introduction to the causal framework and its network description. Section 4 will give an overview of how the tweets, and the data for the prices was collected. Section 5 describes as to how the tweets were pre-processed, analysed for sentiments, given a score; and how the prices were manipulated. Further, some descriptive analysis of the data will be shown as well. Section 6 will apply the Causal Contagion Model (CCM) to find causal links between sentiments and cryptocurrency prices and within sentiments and cryptocurrency prices. Lastly, the findings and potential extensions of this study will be summarised in Section 7 as the conclusion and discussion.

2 Background to Causal Contagion Model

We have heard a lot of statisticians over the years say that "correlation does not imply causation". That is, just because two things are strongly (or weakly) correlated, does not mean that they cause one another, there just might be a hidden common cause behind the two, or not any cause at all, in the case of spurious correlations. In this section, we will go into depth outlining this "cause", and its "effect". But first, will first gain an understanding of what cause and effect are, and then move on to the Causal Contagion Model as a final step.

2.1 Causal Inference

The analysis of a value in isolation is not very interesting, but analysing how that value has an effect on other values can prove to be interesting and fruitful. Therefore, one can want to imagine the effect of an intervention on an object onto another. That is, how will the perturbation in one entity affect the other? An example is how treating a portion of a population with a particular drug affected that portion, compared to the part not treated with that drug. In the context of this project, an example can be how the market would have been if the sentiments of bitcoin were positive at a given point in time (if there exists a causal link from bitcoin sentiments to bitcoin price and further to the prices of other crypto tokens). Another example might be how the market would have been if the price of bitcoin was lower at a given time (given that there is a link from the price of bitcoin to other crypto tokens). In a more general sense, we would be answering the question, "What would the outcome of my actions be if I had done some action 'X', even though I did not commit that action?". We now know what a cause is and, by extension, what an effect is. Cause, in this case, is 'Action X', and the effect would be the consequence of 'Action X'. However, let us formalise it.

"We may define a cause to be an object, followed by another, [...] where, if the first object had not been, the second had never existed" [18] was the definition of 'cause' given by David Hume. Further, hierarchies exist within the causal structure of a problem; they are termed Causal hierarchies. Causal hierarchies are divided into three levels, where each level encapsulates the previous one, the first being 'Association', the second 'Intervention', and the third being 'Counterfactuals' [34]. At the association level, we observe the statistical relations between a set of variables. We use directed acyclic graphs (DAGs, defined later in this paper) to describe these relations. Briefly, a DAG lets us describe said relations as conditional independencies between variables. On the next level, intervention, we interpret these DAGs causally to answer the population-level question of, "What would happen if we did 'X' to a whole population?". We make use of the correspondence of the activity of 'doing' with this level in the form of do-calculus [33] where we see how changing one variable, X in this case, changes (the distribution of) Y [10]. Finally, at the last level of counterfactuals, we have the activity of 'imagining'; these require the most substantial assumptions but allow us to answer individual-level questions instead of population ones [10].

We have seen that the prices of some cryptocurrencies are statistically related to one another in some cases, and in some cases, the price of a cryptocurrency is also related to its sentiment. That is, there can be a case of statistical dependence. When there is a case of statistical dependence between random variables, say X and Y, there exist three possibilities, (1) X causes Y, (2) Y causes X, or (3) there exists another variable, say Z, that causes the both of them and X and Y become independent given Z [39], as denoted by the 'common cause principle'. Given that three possibilities exist when it comes to the dependence between two random variables, it becomes uncertain what causes what. To answer that question, one might intervene independently and notice its effects on these three variables. However, in many cases, it is unethical and seemingly impossible to perturb a random variable. For example, we cannot force a population to take a drug and see its effect.

Causal inference will provide us with the framework to conclude even without an actual experiment, given that certain assumptions are fulfilled (which increase in strength as we move up the causal ladder) [10] what the effect of interventions will be. To answer the causal questions, we will first need to interpret the statistical (in)dependence of the random variables at hand. To do so, as said earlier, we will use DAGs, but before that, we will understand the basic principles of what a graph is, what a graphical model is, and then we can go ahead with understanding what the DAGs are.

2.2 Graphs, DAGs, Graphical Models, and do-calculus

A graph defined as $G(V,E), E \subseteq V \times V$, is a set of nodes (often referred to as vertices as well) and edges, where the edges (if any) connect the nodes to outline some sort of relation between them (A Facebook friend network would be an example of this, where the people are nodes, if they are friends, there exists an edge between them). In the case of directed graphs, the edges go from one node to the other, but the converse is not valid, outlining a one-way connection as opposed to an undirected graph where the connection runs two ways. This bi and uni-directional relationship is described in the figure below which portrays a two-node graph with an undirected (two-way) and a directed edge connecting the two nodes in Figure 1.



(a) Undirected Graph **(b)** Directed Graph

Figure 1. Graph Types

The nodes (will be referred to as a variable as well) in directed graphs can have edges directed at or from them. For any node 'i' in a directed graph, we define node 'j' as the parent of node 'i' if 'j' has an edge directed at 'i'. In Figure 1, we see that A is the parent of B, formally, $pa(B) = A$, where $pa(i)$ represents the set of parent any given node 'i' has. A, by extension, is the child of node B, $ch(A) = B$, where $ch(i)$ represents the set of children any given node 'i' has.

When talking about directed acyclic graphs, we will consider the case of directed graphs, as the name suggests. A graph can have cycles within it. That means we can start from a given node and end up on the same node following a path. However, in the case of DAGs, we prohibit that, hence the 'acyclic' in its name. One can look at it in an intuitive sense as well in terms of the causal graphical framework whereby, if a cycle exists, we can infer that any given node in that cycle is causing itself, which seems counterintuitive.

We said earlier that we could use DAGs to visualise statistical (in)dependencies between variables. DAGs have been around for a long time and have been used to model statistical data. One of the first uses of DAGs was in 1921 by Sewall Wright [49]. One of the main reasons for the use of DAGs is the fact that there exist many efficient algorithms are used for the calculation of conditional distributions; in our case, we will model causal dependencies, which are also very easily modelled by DAG. The graphical structure provided to us by the DAG has a natural causal interpretation to the data which is very useful in modelling the interaction between different nodes.

Using these concepts in graphs, one can represent many problems in the real world, that is, create a graphical model for a problem of choice (Examples include Social Network Analysis, amongst other things). In our case, "a graphical model is a family of probability distributions defined as a directed or undirected graph. The nodes in the graph are identified with random variables, and joint probability distributions are defined by taking products over functions defined on connected subsets of nodes" [21]. To put this idea more concretely, in our case, given a directed graph $G(V,E)$, a directed graphical model X relative to G is defined by a vector of random variables $X = [X_1, X_2 \ldots X_{|V|}]$ where V refers to the vertices in a graph. This means that every random variable X_i is associated with a corresponding node V_i in the graph G.

We further state the probability density (or probability mass) function of the graph in Equation(1), which states that the joint probability distribution factorises as the product of the probability each node in the network conditioned on its parent satisfying the Local Markov property which states that the nodes in a directed network (DAG, as explained later) are independent of their non-descendants given its parents, and also the Minimality assumption

which states that adjacent nodes in the network are dependent.

$$p_X(x) = \prod_{v \in V} p(x_v | x_{pa(v)}) \tag{1}$$

We would be working with 'Causal Graphical Models', therefore, we define the causal GM to be a directed graphical model with the additional assumption (a causal interpretation) which is stated in Equation (2):

$$p_{X/i}(x_{X/i} | do(X_i = x_i)) = \prod_{v \in V \wedge V \neq i} p(x_v | x_{pa(v)}) \tag{2}$$

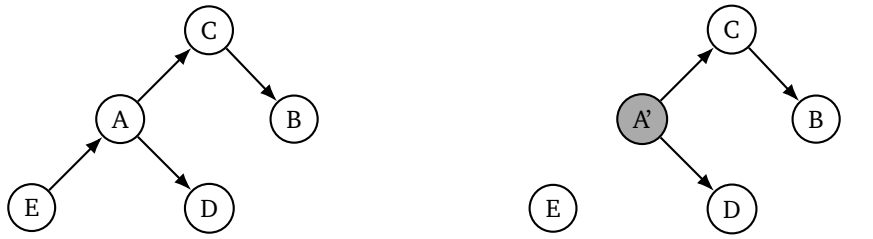
Here, we are imposing the value of X_i as some arbitrary value x_i representative of its distribution. This corresponds to the act of 'doing', that is, intervening in the data and asking what the value of some X_j be if $X_i = x_i$. This can be interpreted as what the price and sentiment of the other cryptocurrencies in our network be if the price of bitcoin is set to some number (ideally representative of bitcoin prices). This act of imposing the value of a random variable X_i to be some constant x_i is referred to as the do-calculus [33] which was introduced by Judia Pearl in the year 1995 as a way to facilitate the identification of the causal effects in non-parametric models (models which do not conform to a normal distribution) [32].

Log-return data can be modelled as a non-parametric model as it is suggested that the characteristics of log-returns do not conform to normality [46]. Therefore, we can use do-calculus to model the effects of the causal intervention on an asset in the causal graph on another asset. Therefore, to measure the effects on a random variable Y, through intervention on another random variable in the graph X would be as represented in Equation (3).

$$p_Y(y | do(X_i = x_i)) = \sum_{x_1} \sum_{x_2} \dots \sum_{x|X/i|} p_{YX/i}(y x_{X/i} | do(X_i = x_i)) \tag{3}$$

Equation (3) calculates the marginal probability distribution of Y after the intervention over the probability mass function (or probability density function) of the Causal Graphical Model as defined in Equation (2).

After the intervention onto the graph is complete, the graphical interpretation of the Equation (2) leads us to an updated graph whereby all the incoming edges to the nodes onto which we are intervening are cut, resulting in a new DAG incorporating all the dependencies from the previous graph but not the dependencies of the variable intervened upon. This is because when we intervene in a variable, we assume that the value of the variable is determined only by the intervention, not by any other factors [10]. These concepts are better understood using graphs. Therefore, a pictorial depiction of intervention is described in Figure 2.



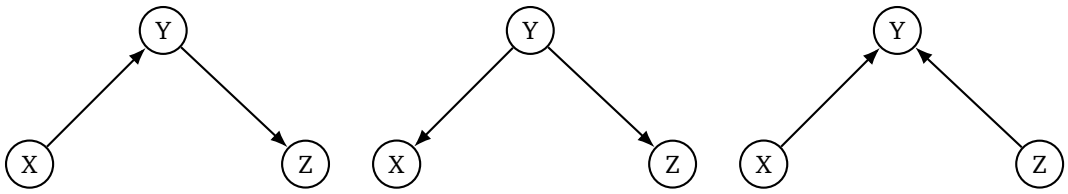
(a) Before Intervention **(b)** After Intervention

Figure 2. Intervention on A

As we know, the **'do'** operator sets the value of a variable, thereby disregarding all the effects any other variable has on the one we intervened on. Visually, all the incoming edges have been cut as we do not want any other factors affecting the intervened variable. Figure 2 shows the effect on intervention on the original DAG. We refer to the DAG resulting from the intervention as the manipulated DAG. Note how we intervened on A (A' after intervention), and it cut the incoming causal arrow from E.

2.3 From Directed Acyclic Graph to Structural Causal Model

We are going to be monitoring causal (in)dependencies from the DAG that we obtain in order to model the market. A simple example of how the dependencies between the nodes would flow can be better understood in the form of an example. Let us consider a two node graph $G(V = \{X, Y\}, E = \{(X, Y)\})$. Here, node X has a direct causal effect on Y. Now, if we want to introduce another node into the graph, say node Z, and associate it with any (or all) nodes in the network, there exist only three ways to do so. The first is lining the nodes in a chain. Second, the first node act as a fork or a common cause. Furthermore, third, having the third node act as a cause to the second node along with the first one [29]. The depiction of all three is given in Figure 3.



(a) Chain **(b)** Fork **(c)** Immorality

Figure 3. Directed Acylic Graph

In the case of chains and forks in Figure 3, we can see that X and Z are dependent. In chains, Z is dependent on X through Y, and in the case of forks, we can see that changes on Y would result in changes in both X and Z. That means they are associated with their common cause (Y). They share the same set of independencies as well. That means if conditioning on Y on both graphs, we would block the flow of dependencies. This can be better understood by considering the Local Markov Property we discussed earlier, whereby any given node only depends on its parents. This means that conditioning on Y would block dependence from Y to Z, thus rendering Z independent of X.

The blockage of dependencies can be attributed to the simple formula for conditional independence $p(X,Z|Y) = p(X|Y)p(Z|Y)$ which can easily be proven by noticing that $p(X,Z|Y) = \frac{p(X,Y,Z)}{p(Y)}$. We know the factorisation of the joint probability of the network from Equation (1); putting that into what we got earlier would yield the following in the case of chains:

$$p(X,Z|Y) = \frac{p(X)p(Y|X)p(Z|Y)}{p(Y)}$$

We know from Bayes theorem that the equation can be rewritten as the following:

$$p(X,Z|Y) = p(X|Y)p(Z|Y)$$

Thus proving that conditioning on Y leads to the independence of X and Z. The same can be done in the case of forks as well. The more interesting case, however, is the case of immorality. In this case, two nodes X and Z have a direct causal effect on Y, but $X \perp\!\!\!\perp Z$, which can be confirmed by the fact that these two nodes do not share any causal links from one node in which that node becomes the common cause, nor are they associated like nodes in a chain. Thus, they are two seemingly unrelated events which have a causal effect on a common variable. In this case, Y would be called a 'collider'. Interestingly, conditioning on a collider makes the parents of the collider dependent [10][29] due to the fact that one can infer about node Z if it has the information of node X because conditioning on Y leads to us having the direct causal information of the node Z, and therefore its causal effects can be computed.

One idea that stems from this causal dependency structure is the idea of a Markov Blanket. It was first introduced in 1988 by Pearl in his book 'Probabilistic reasoning in intelligent systems: networks of plausible inference' [30]. A Markov-Blanket is a central topic in the study of Bayesian Networks such as the one that we are studying. A Markov blanket of a node is given by its children, spouses, and parents. A Markov blanket has an interesting property which states that given a Markov blanket for a node, all other nodes in the network are independent of that node. In

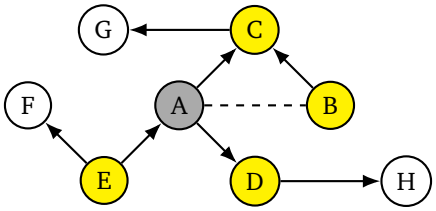


Figure 4. Markov Blanket Example

Figure 4, we are given an example of a Markov Blanket around node A. Here, the nodes in yellow represent the Markov Blanket around A, and the white nodes represent the nodes that are not in the Markov blanket. Notice that the nodes A and B are connected through a dashed edge. That means that node B is the spouse of node A.

When we have extensive networks (graphs), it would be challenging to manually check if two nodes are independent. To mitigate this problem, we can use an algorithmic tool called d-separation [14]. The specifics of this tool are out of the scope of this report, but it is an independence model defined on graphs, which states assumes the Markov Property of $X \perp\!\!\!\perp_G Y|Z \Rightarrow X \perp\!\!\!\perp_P Y|Z$ [10], stating that if two nodes are d-separated in a graph, then that means that they imply conditional independence between themselves given a set of nodes Z. There are two ways in which one might learn the DAG of a problem; the first is causal discovery [44], where one learns the DAG from the data. The other is a Structural Causal Model (SCM) based on the theory and understanding of the problem [10].

A Structural Causal Model is based on the Structural Equation Model (SEM), which can be used to depict the propagation of dependencies (effects) amongst the variables (nodes in a graph) [31]. Formally stating, we can say that SCM is a tuple of (1) a set of endogenous variables V, (2) a set of exogenous variables **U**, and (3) a set of functions F used to generate each endogenous variable with potentially other variables as the parameter to the function [29].

A nice property of an SCM is the fact that the derivation of a DAG from an SCM is quite straightforward due to the fact that we have all the causal relationships already defined in an SCM, and therefore, DAG is just a matter to visually interpret an SCM. An example of SCM would be the following:

$$\begin{aligned} X &:= \mathbf{U}_x \\ Y &:= f_Y(X, \mathbf{U}_y) \\ Z &:= f_Z(Y, \mathbf{U}_z) \\ W &:= f_W(X, Y, \mathbf{U}_w) \end{aligned}$$

The corresponding DAG from these set of equations is depicted in Figure 5.

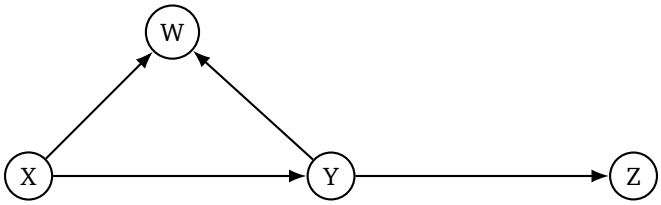


Figure 5. DAG from SCM

Since we have established that a DAG can be derived straight from an SCM; therefore, the interventions in an SCM are pretty trivial to perform as well, whereby we set the structural equation of a given variable that we plan to intervene on to the value we want to set replacing the function to generate that endogenous variable ($T := t$ instead of $T := f(A, B, C\mathbf{U}_T)$). SCMs relate causal and probabilistic statements, which are vital for our study later. We would have little knowledge about the problem at hand; therefore, we may have to interpret the DAG from the data given to us. To do so, we make use of the PC algorithm.

2.4 The PC Algorithm

In our study, we analyse the causal links between the prices of different cryptocurrencies, within the sentiments of the crypto tokens, and between the sentiments and prices of the said token(s). Several algorithms can be used to analyse these links (in the form of a graph), in addition to the PC algorithm, which is a constraint-based algorithm that looks for conditional independence between nodes [43]. Another type of algorithm is score based algorithm, which maximises a causal objective function [40]. The PC algorithm employs some steps to infer the graph from data. These are the following:

2.4.1 Inferring the Undirected Graph

At first, we start with a complete causal graph of undirected edges. That means that every random variable is connected to each other.

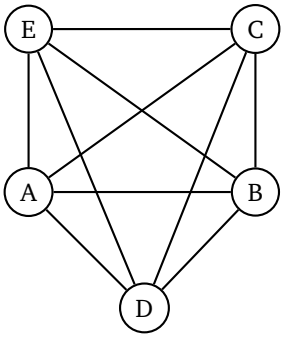


Figure 6. Complete Graph from Data

After all the random variables are interconnected, the algorithm will look for the true graph (DAG) that can be produced from the data. To do so, it removes the edges between pairs of variables by checking if the pair is conditionally and unconditionally independent. It removes an edge if it finds that any node in the network is unconditionally independent of some other node. To check for (conditional) independence, it will check if the two variables are independent conditioned on a set of variable(s) [15]. This set, say set S, grows iteratively until we exhaust all adjacent nodes or until we find the set S such that the two variables say A and B are independent given S, $A \perp\!\!\!\perp B|S$. If the size of the set after checking for the independence is greater than 0, then we know, that we are supposed to remove the edge. After we are done with this step, we are left with a graph of nodes connected with undirected edges. This graph that we left with is called a skeleton.

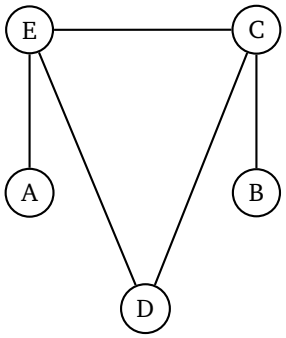


Figure 7. Skeleton after removing edges.

The next step in the algorithm is the orientation of the edges to form causal links and then have a visual idea as to what influences what.

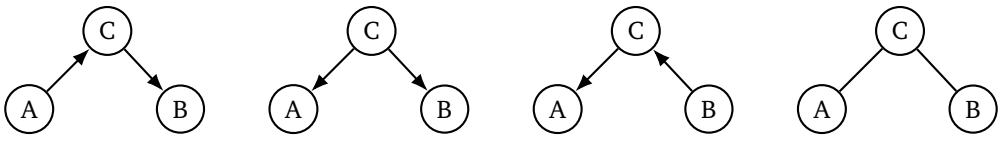
2.4.2 Orientation of the edges to form Causal Links

We know that for each pair of nodes that is not connected by an edge, there exists a set S (as defined in the section above), which contains nodes on which, if the two nodes which are not connected are conditioned upon, will lead to the independence of the two nodes. If this set S does not exist, we can assume that the two nodes are unconditionally independent.

We then try to follow some orientation rules, some of which are (i) if the two nodes are connected to a common neighbour. If they are, and if the neighbour is not in the conditioning set S, then we can orient the edges from the two nodes towards the neighbour, to form a collider structure (a depiction is shown in Figure 3, termed as immorality), also known as the v-structure. The concept of Markov Blanket is central when it comes to finding the colliders due to the inference of spouses from the children. (ii) If there is a link from node A to node B, and there is a link (undirected) between node B and node C, such that A and C are not adjacent, then we orient the edge between B and C and make it directed from B to C, also known as 'orientation propagation' [15]. (iii) If three nodes A, B, and C have directed edges that connect A to B and B to C, and there is an undirected edge between A and C, then, in order to avoid a cycle, this edge is oriented away from A, that is, the edge becomes a directed edge from A to C, where A is referred to as the common cause between B and C. Many other rules are also present in the algorithm, which are defined in many studies, one of which was conducted by Meek [27]. We only defined three orientation rules as they capture the three cases in which we said a dependency could be added to a graph which is depicted in Figure 3; that is, we defined the orientation in the cases of chains, forks, and colliders or immoralities.

At the end of the orientation, the algorithm ensures that there were no disrupted links that existed before and that there was no introduction of a cycle (as can be inferred from the acyclic in Directed Acyclic Graphs). There, however, is a catch; not all undirected edges can be directed following the rules, resulting in the graph having some edges that are not directed. This will result in a graph that is called a Complete Partially Directed Acyclic Graph (CPDAG). This occurrence of the CPDAG can be attributed to a small number of nodes present in the network. Therefore, the higher the node, the easier it is to orient the edges. CPDAG represent a Markov equivalence class of all DAGs encoding the same conditional independencies [5].

Therefore, a given CPDAG can have several equivalent DAGs. An example would be when there are V-structured nodes but not in the form of a collider. This would result in three possible DAGs. First, we can have a common cause. Second, we can have A causing B, then B causing C. Lastly; we can have C causing B, which causes A. This is depicted pictorially in the figure below, where we show that the graphs a), b), and c) are Markov equivalent to d); they all encode the same conditional independence structure, where $A \perp\!\!\!\perp C|B$.



(a) $A \perp\!\!\!\perp B|C$ **(b)** $A \perp\!\!\!\perp B|C$ **(c)** $A \perp\!\!\!\perp B|C$ **(d)** $A \perp\!\!\!\perp B|C$ in a CPDAG

Figure 8. CPDAG and its equivalent DAGs

We can now look at how it all connects to the proposed solution, where we are going to use the PC algorithm with an added step to make it more stable. But before that, we are going to look at a way to preprocess the time series data in order to fit it into the framework.

2.5 Normality and Copulas

The normal (or Gaussian) distribution is a very special distribution when we talk about the realm of probability and statistics. In nature, a person's height, weight, and test scores are normally distributed. Many statistical calculations can be made more accessible if we know that a normal process generates any given data. It is also an excellent assumption to make given that the mean of any set of variates with any distribution having a finite mean and variance tends to the normal distribution [48]. In the world of finance, many use log-returns of instruments to model the market, which they believe to be normally distributed so that they can take advantage of the normal distribution and its properties. However, it is shown that it might not be the case that log returns are normally distributed [46] [38]. Thus, in order to mitigate the effects of the non-normality of the data at hand, and take advantage of the properties of normality; we will transform our data into the data generated by a normal latent process using copula transformations to capture the non-gaussian behaviour of the log returns.

We assume that the data we have, say X_{it}, where i refers to what asset (bitcoin, Cardano, etc.) we are talking about, at time t, follows a distribution F_i. We further conjecture that a latent multivariate normal variable $Z_{it} \sim N(0,1)$ exists. We state that the observed data X_{it} is generated by transforming Z_{it}.

$$X_{it} = f_i(Z_{it}) \tag{4}$$

The transformative function is protrayed as f_i in Equation (4). The function would specifically be:

$$X_{it} = F_i^{-1}\overbrace{(\underbrace{\Phi_{0,1}}_{\text{cdf of N (0,1)}}(Z_{it}))}^{\text{Un (0,1)}} \tag{5}$$

In Equation (5), we do not conjecture on the distribution of F_i, we simply use the cdf from the observed data. However, we want to have our data transformed to the true distribution. In order to do so, we would just manipulate Equation (5) to get the data values in the latent (true) distribution:

$$Z_{it} = \overbrace{\Phi_{0,1}^{-1}}^{\text{inverse cdf of N (0,1)}}(\underbrace{\hat{F}_i}_{\text{Estimated Distribution}}(\overbrace{X_{it}}^{\text{Observed Data}})) \tag{6}$$

This would then yield us the values of the latent distribution, which are copula transformed and are normally distributed, unlike log returns. This can be used in price prediction by using our framework on Z_{it} and then transforming the predicted data that the framework will generate back to the distribution F_i using Equation (5).

We can now apply this transformation to actual data and look at its effects. First, we will look at how the time series is transformed, and then we will look at how the distribution of a token changes after applying the copula transformation. In this case, we will look at the log-returns of Bitcoin for the visual depiction. From Figure 9, we can see that before applying the copula transformation, the distribution looked non-normal, with long tails and some outliers. However, after the transformation, all of it was regularised, and we got a lovely bell curve. The same can be said for the time series data. We can see that before the transformation was applied, there were some outliers when there were some market volatility periods. Nevertheless, after the transformation, the time series is more regular; therefore, there is a potential for establishing trends within the data. We can further, compare and contrast the different distributions, and establish independence, amongst other things, given that the distributions will be normalised after this transformation is applied.

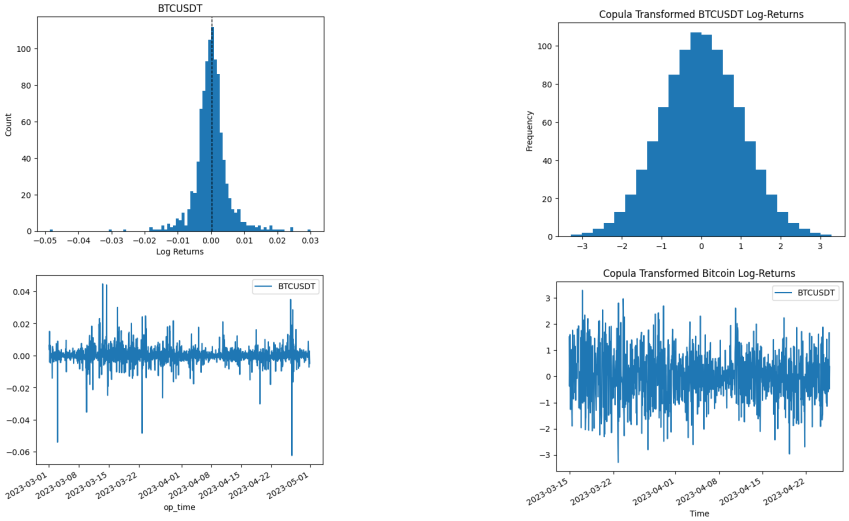


Figure 9. Effects of Copula Transformation on Bitcoin Log-Return Distribution and Time-Series

We now have data values from the latent distribution which follows the normal distribution, but in order to check that the data is indeed generated by a normal distribution, we employ a method to test for the normality of the data. The method that would be used in this study will be the Jarque-Bera (JB) Test for normality [20]. It is essentially a Chi-squared test with two degrees of freedom. The JB test is defined by:

$$JB = \frac{n}{6}\left(S^2 + \frac{(K-3)^2}{4}\right) \tag{7}$$

Here, S refers to the skewness of the data, and K refers to the kurtosis of the data. The specifics of this test are outside the scope of the study; however, it is described in depth in [20] and [45]. We will primarily concern ourselves with the p-value obtained by this test. If the p-value is less than 0.05, we can conclusively reject the null hypothesis (H_0), which states that the data comes from a normal distribution. Therefore, if we have p-values closer to 1, we can conclude that the data is normal.

3 Introduction to the Network Description of Contagion

Contagion refers to the "significant increase in comovements of prices and quantities across markets, conditional on a crisis occurring in one market or group of markets" [35]. Studies had identified that when the Cryptocurrency Market was relatively new, it was not as prone to contagion and therefore was a very good diversifying strategy [3]. There are many definitions of contagion, but the one quoted above is very close to what we consider a contagion is in our studies. For us, contagion would be the changes in price due to factors spreading from instruments to instruments (in our case, crypto tokens) that cannot be explained by individual trends, often referred to as "pure contagion" [40] [47]. Our approach is based on the framework presented by Rigana [40], whereby, from the combination of the causal networks with a structural Vector Autoregressive model, we can extract from the log-returns of the cryptocurrency prices as to what the token itself causes part of the change in prices, and what part is caused by contagion effects from other tokens.

That study, however, does not take into account investor sentiment. Therefore, we will be extending the use of that framework to encode the values of sentiments and see how the prices of the cryptocurrencies change when paired with other currencies and their sentiments from a causal perspective. Therefore, we would have eight crypto tokens in our graphs with the log returns of their prices and eight sentiments corresponding to each crypto token, totalling 16 nodes, and then we try to see the causal links to see what causes what. We analyse this graph, or the complete graph as defined above, by using the PC algorithm. This study then uses the SCM with a network-based contagion measure, combining a vector autoregressive component and a network contagion component as defined below [40]. We will be using the copula transformation on the data from our nodes; therefore, the equation will change slightly.

$$\underbrace{Z_{i,t}}_{\substack{\text{Copula Transformed Log-Return and Sentiment on token 'i'}}} := \alpha_{0,i} + \underbrace{\sum_{l=1}^{L} \alpha_i^l \phi_{\mu,\sigma}^{-1}(\hat{F}_i(X_{i,t-l}))}_{\text{Autoregressive Component}} + \underbrace{\sum_{j \in pa(i)} \beta_{ji} \phi_{\mu,\sigma}^{-1}(\hat{F}_j(X_{j,t}))}_{\text{Contagion Component}} + \underbrace{\epsilon_{i,t}}_{\text{Error Term}} \tag{8}$$

Equation (8) provides us with the evolution of the log returns and sentiments of the crypto token 'i', through time 't' represented by $Z_{i,t}$, l are the number of lags that are considered for the problem in the AR part, α_i^l are the AR coefficients at lag l for a crypto token i, and β_{ji} are the causal effect of node j on node i, where $\beta_{ji} = \frac{d}{dx_j} E(Z_{i,t}|Z_{-i,t}, Z_{i,l})$ as defined by the causal literature, which states that "the causal effects are defined by the partial derivates of the expected log returns" [40]. Notice that the sentiments and log-returns are copula transformed, and F and ϕ have the same meaning described in Section 2.5. This can then be simplified in a compact form from the structural form in Equation (9) as follows:

$$Z_{it} := \alpha_{0,i} + AR_{it} + NECO_{it} + \epsilon_{it} \tag{9}$$

Here, AR_{it} refers to the autoregressive part of the equation, and $NECO_{it}$ refers to the network contagion part of the equation.
Now that we have the SCM, the PC algorithm can move forward and analyse the causal structure from the available data. In case the PC algorithm produces a CPDAG, a subset of Markov equivalent DAGs that can explain our data, we will also find a multiset of possible NECO estimates $\hat{\beta}_{ij}$. After the end of the PC algorithm, we need to further estimate the values of β_{ji} from the resulting DAG. After the calculation of the NECO, we can then calculate something known as the NECOF (Network Contagion Factor) which would give us the impact of contagion on individual assets [40]. This framework internally copula transfroms the data given to it and when giving the result, it transforms it back to the real distribution. We however just focus on building and analysing the graph that is outputted by the PC algorithm.

4 Data Collection

For the purpose of this project, there are two different types of data classes that we needed to consider, these are, the data relating to the price of a given crypto token over time, and the data relating to the sentiments of the same crypto token over the same period of time. We will first talk about the data collection relating to the sentiments of the different tokens. See `https://github.com/lov-1234/BachelorProject2023` for the code.

4.1 Twitter Data Collection

A wide variety of information is exchanged on Twitter, with over 6,000 tweets per second [41]. This, therefore, turns out to be the best source of user sentiment information about a particular topic, more so than other surveys, newsletters, or sentiment databases that would not provide us with almost instantaneous happening on a given topic, as mentioned before. We utilised this information and made use of the Twitter API (see `https://developer.twitter.com/en/docs/api-reference-index`), and in the case where the use of the API was not feasible, the use of web scraping techniques was made. Simple HTTP requests from the requests module of Python were used to interact with the Twitter API. However, as said before, in some cases, using an API was not possible. In those cases, we used Scrapy, which we will describe later.

4.1.1 Fetching Tweets Using API

The API is the simplest and fastest way to collect tweet data from Twitter, from what we found out. The endpoint "`https://api.twitter.com/2/tweets/search/recent`" would fetch all the recent tweets within a week. However, there is a rate limit of 450 requests per 15-minute window [11]. And a limit on the tweets that can be fetched within a month (this usually depends on the version of API that one is using), which was 500,000 in our case. This severely limited amount of tweets can be fetched to establish any correlations between the prices and sentiments since the sample space of the data collected for sentiments was severely limited. There were many iterations to make the module fetch the tweets from Twitter. At first, we started with collecting the tweets towards the end of the day to get the tweets about the closing sentiments of the users relating to a given cryptocurrency. A sample query for fetching the tweets for the keyword 'bitcoin' for the hour 24th April 2023 at 11:00:00 UTC to 24th April 2023 at 12:00:00 UTC, having the information of the language of the tweet would be of the following form:

```
https://api.twitter.com/2/tweets/search/recent?q=bitcoin&start_time=2023-04-24T11:00:00z&
    end_time=2023-04-24T12:00:00z&tweet_fields=lang
```

We hypothesised that the tweets collected towards the end of the day are more representative of the price of a given cryptocurrency. This is because the price of a token changes throughout the day since the market is open 24 hours. Therefore, the tweets at the start of the day represent the prices at the start, and the tweets towards the end of the day represent the prices towards the end of the day.

This led us to a problem whereby we could only look into the tweets within a week using the API, meaning we would have very little data to work with. Therefore, we chose to reduce the intervals from daily to hourly, increasing the granularity and enabling us to analyse the data over many data points. At first, we collected about 500 tweets per hour per token. However, that meant that in the cases of some popular tokens, which have a higher tweets frequency than others, 500 tweets per hour starting from the end of an hour and moving towards the start of the hour would only cover about 5 to 10 minutes of data before the closing of that hour which might not be representative for modelling the prices for the next hour.

Another hypothesis that can be made is that the closer the tweet is towards the end of the hour, the more it is possible that we see the effects of the tweets in the following hours. That means that if we have a tweet that has a positive sentiment regarding the price of bitcoin towards the end of the hour, then the potential effect of the tweet can be prompting the other users to invest in bitcoin, thereby increasing the price of bitcoin, will be seen in the following hours because it is too close to the end of the hour. However, it might have some effect on the current hour as well. Due to this fact, we had to take in more tweets during an hour, which would encapsulate the effect of the user sentiments about a crypto token during a particular hour and mitigate the effects of the tweet that was made close to the end of the hour.

After considering both hypotheses, we can move forward with sending the requests. To do so, we had first to form the query. As said above, we used the requests module in Python to form the requests. Forming the query in the requests module would be done by providing the URI of the address we want to send the request, the authorisation tokens, and the query parameters.

```
Sample Query:

response = request.get(https://api.twitter.com/2/tweets/search/recent, authorization=*, params={
    "query" : "bitcoin",
    "start_time" : "2023-04-24T11:00:00z",
    "end_time" : "2023-04-24T12:00:00z",
    "tweet.fields" : "lang",
    "max_results" : 100
})
```

The piece of code above sends a GET request to the Twitter API, which encodes the same information as the example query given above, but there is another field added, which is 'max_results'. Twitter API lets us retrieve a maximum of 100 tweets per request. The response to the request sent above would have the data in the form of tweets that we retrieved and a pagination token in the case of the query having more than the 'max_results' tweets in a given period. We can then retrieve the pagination token from the request and then keep on collecting the tweets until we do not have any more tweets left for the given period or we reach the threshold for the tweets that can be collected for the hour (which is set as 4000 in the file '**enums.py**').

After we have fetched the tweets, we can filter out the tweets that are not in English because our sentiment analyser only analyses the text if it is in English. To do that, we check if the tweet's language is English by just checking the retrieved language associated with every tweet object, as we requested the tweet's language when sending the request to Twitter (check field 'tweet.field' and the mapping 'lang' in the request object).

Further, many bots keep posting tweets which promise the user 'earn a reward', 'PayPal some money and earn bitcoin', amongst other things. We do not want such data to cloud the basic sentiment of the day. To remove that, we can make a heuristic whereby we can match some words (named 'WORDS_FOR_REMOVAL') frequently occurring in the bot tweets to the tweets we retrieved and remove the tweet if a word matches the words for removal.

```
WORDS_FOR_REMOVAL = {
    'retweet', 'earn', 'reward', 'paypal', 'giveaway', 'venmo', 'cashapp'
}

def filter_english_results_and_filter_bots(tweets):
    return [(i["id"], i["text"]]) for i in tweets if
            i["lang"] == "en" and len(set(i["text"].lower().split()) - WORDS_FOR_REMOVAL) == len(
            set(i["text"].lower().split()))]
```

Here, we use the concept of set difference by checking that when we subtract the set that is the 'WORDS_FOR_REMOVAL' from the set that is made by splitting the tweet into the spaces, we get a difference in the number of words or not. If we get a difference, then that means that the tweet contained some of the words that we did not want to have in a tweet because that would mean that it would be a tweet made by a bot according to our heuristic and, therefore, should be removed.

The API, however, has a flaw. As outlined above, we can only send a fixed number of requests to the API in a given amount of time (15 minutes in our case). Due to this, we have to limit the number of requests we send per

15-minute window. If we do not cap the requests, we will not get any response. To mitigate this fact, we can put the thread of execution to sleep once we have passed the threshold of the tweets allowed in a 15-minute window.

```
TIME_WINDOW = datetime.timedelta(minutes=15).total_seconds()
SOFT_REQUESTS_LIMIT = 420
...
<REQUEST SENT AND RESPONSE PROCESSED>
REQUESTS++
if REQUESTS > SOFT_REQUESTS_LIMIT:
    sleep(TIME_WINDOW)
    reset_requests_counter()
...
```

In the code above, everything happens in a loop, and only part of the loop is visible where the logic of the request capping is implemented. After we send the request and filter the results, we increment the request counter by one, which is set to 0 before the first request is sent. Once we pass the soft threshold, we put the thread of execution to sleep for 15 minutes and then reset the requests sent to 0 to start a new session with the API in which we can begin to request the tweets again.

The only problem with the API and its usage is a cap on the monthly tweets retrieved, as outlined above. Reaching the request threshold would result in our service being unable to fetch the tweets from the API anymore, resulting in a discontinuity of the data collected for sentiment analysis. One way of increasing this limit can be applying for elevated access for the Twitter API, whereby one can fetch up to 2 million tweets per month, and one can also search for tweets back to the beginning of Twitter. We chose a different approach to mitigate the glaring issue of the Monthly Tweet Cap.

4.1.2 Twitter (Nitter) Scraping Using Scrapy

We use Scrapy to scrape Nitter (instead of Twitter) to mitigate the issues the Twitter API faces. This would lead to us having the tweets over a more extended range of dates. We use a self-hosted instance of nitter using docker containers because it is a lightweight alternative to Twitter and also because Twitter cannot be scraped using the standard scraping techniques, as they removed this feature to fend off scrapers quite recently. Nitter provides essentially the same functionality as Twitter, which is perfect for this project.

Scrapy was pivotal for this part of the project as it allowed us to scrape the web pages in a very structured format. The code for the same can be found under the directory '**scrapy_twitter**' and in the file '**twitter_crawler**'. The object yielded by the spider at the end of scraping a tweet would have three fields: 'tweet', 'token', and 'date'. However, the structure of the webpage that displays the tweets in nitter is terrible because there are links embedded in the 'div' tags, and the 'div' tags are without the 'p' tags, which results in commas between every link in the tweet and every tweet itself. Say, for example, if there is one tweet that says 'Today is a very good day for #Bitcoin but a bad day for #Dollar', it would have two links in the hashtags, and when scraped, the resultant text would be of the following format:

```
retrieved_list = ["Today is a very good day for", "#Bitcoin", "but bad day for", "#Dollar"]
```

We would keep scraping and append all the tweets from the main page to this retrieved list. However, this would not have a set structure as the tweets would be separated by commas. This, therefore, makes the distinction between tweets very hard. In order to mitigate that, we look at the link of the tweet when scraping and then redirect the spider to the link of the tweet so that it can scrape the entire tweet without the issues of non-identification (differentiability of one tweet from the other) that we faced before during the scraping of the homepage.

```
tweet_links = response.css('.tweet-link').xpath('@href').getall() # Gets all the links to redirect the spider to
for tweet in tweet_links:
    yield response.follow(tweet, callback=self.parse_tweet, cb_kwargs={
        'token': token
    })

def parse_tweet(self, response: scrapy.http.Response, token=None):
    tweet = ''.join(response.css(".main-tweet .tweet-content ::text").getall())
    date = pd.to_datetime(response.css(".tweet-published::text").get(), format=TWEET_DATE_PARSE_FORMAT).replace(
        minute=0)
    is_bot = False
    for i in WORDS_FOR_REMOVAL:
        if i in tweet.split():
            is_bot = True
```

```
    if not is_bot:
        yield {
            'tweet': tweet,
            'token': token,
            'date': date
        }
```

After the spider is redirected to the individual tweet, it scrapes it, and then when we have the list as shown above, we join every element and yield the result as one string. This lets us separate the tweets, and it is a better choice than scraping the main page resulting from the query. We then filter the tweets with the same heuristic as the one we used before, whereby we check if the word for removal exists in the tweet, and if it does, we remove it. Notice that we check for the presence of bot tweets according to our heuristics and do not filter the languages. This is because our queries are constructed in a way in which we only get English tweets from nitter.

After fetching the tweets, we have following data available for each token since 2023-03-14 at 21:00:00 UTC to 2023-04-25 at 19:00:00 UTC after capping the tweets per hour to 4000 per token:

Token	Number Of Tweets
Bitcoin (BTC)	1'489'071
Polygon (MATIC)	155'425
Ripple (XRP)	281'220
Dogecoin (DOGE)	134'882
Cardano (ADA)	131'504
Polkadot (DOT)	24'533
Hedera (HBAR)	23'101
Solana (SOL)	237'982
Total	2'477'718

Table 1. Tweet Data

4.2 Binance Price Fetching

Binance is the largest cryptocurrency trading platform in the world [36]. Due to that, and that we can easily fetch the market data in terms of volume, and prices, amongst other things relatively easily, we chose this exchange to fetch the price information of crypto tokens in this project. The API is very straightforward; however, there is a service that is implemented by Binance itself (see `https://github.com/binance/binance-public-data`) or the directory '**historica_data_binance_fetching_service**' of this project, that can be used to fetch much information regarding a given crypto token.

After installing all the dependencies following the instructions as given in the README.me file in the folder. For our purposes, the command '`python3 historical_data_binance_fetching_service/download_klines.py -t spot -s BTCUSDT HBARUSDT SOLUSDT MATICUSDT DOGEUSDT XRPUSDT ADAUSDT DOTUSDT -i 1h -startDate 2023-03-13 -endDate 2023-04-25`' would fetch the data related to all the cryptocurrencies in this study with its trading pair, which is USD Tether (a stablecoin with its value pegged to the US Dollar).

Execution of the command above results in us having many zip files in a particular directory that can be specified as a command line argument to the price fetching service. To unzip the zip files fetched from Binance, we can call the function '**unzip_all_data**' in the file '**Reader.py**' by passing the path where the zip files are kept and where we want the extracted data to be. We can further form dataframes from the extracted data of the prices; all we need to do is call the function '**get_global_dataframe**' in the file '**PriceAggregator.py**' which takes in the path where the extracted CSV files reside, and how we would like to orient the dataframe we are creating. This would yield us a dataframe of the prices of the crypto tokens, which can then be processed however we want. In this study, we transform the prices to log-returns and then copula transform them to model the market.

5 Exploratory Analysis and Summary Statistics

In this section, we will look at how we pre-processed the data for the sentiment analysis, how we analysed the sentiments, how we processed the data for the prices with some focus on the trends relating to the sentiments.

5.1 Pre-processing and Sentiment Analysis

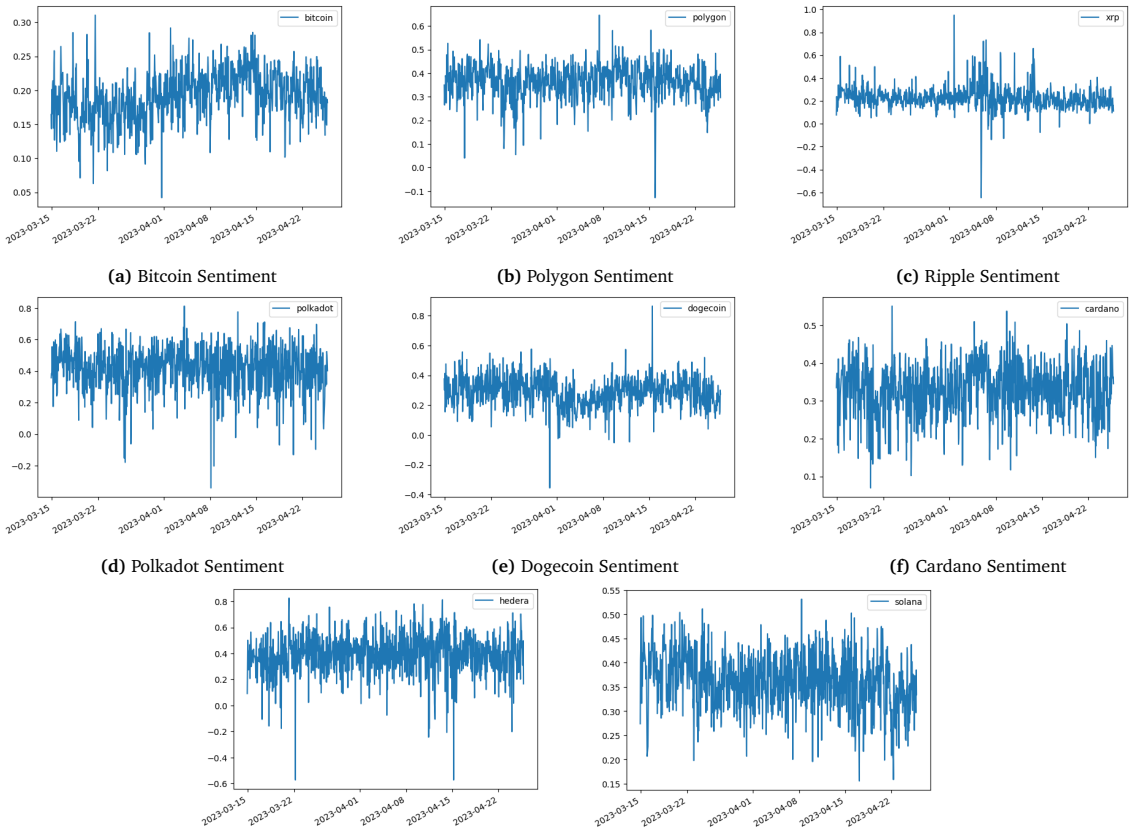
To score the tweets based on their sentiments, we use the Valence Aware Dictionary and Sentiment Reasoner (VADER) algorithm [19]. The tweets will either be classified as positive, negative or neutral. These signals would signify the Investor Sentiment at a given time, whereby the positive sentiment of the investor would result in the investor believing that the price would increase in the future. The VADER algorithm is a lexicon and rule-based sentiment analysis model suitable for analysing the sentiments expressed on social media. It has also been shown that this algorithm is better than both humans and other classifiers [19]. VADER computes a "normalised weighted composite compound score between -1 and 1" [22]. This score indicates the positivity (≥ 0.05), negativity (≤ -0.05) or neutrality of a tweet (≥ 0.05 and ≤ -0.05) [19]. After giving each tweet a score, these scores are aggregated into a time series with hourly intervals. Furthermore, we take the mean of the score per hour to represent the population's sentiment towards the token during that hour. We do not take into account the public metrics of the tweets. That means we do not consider how often a tweet has been liked or retweeted or whether a tweet is part of a conversation. That means the tweet is a standalone entity in the context of this study.

The VADER algorithm needs the data structured in a particular way. That means we cannot pass the raw tweets to the sentiment analyser. Let us take a tweet as an example (this tweet may or may not exist, it is just an example), "This is a #very #good day for #bitcoin and #crypto in general". This type of pattern is widespread while tweeting, whereby users can add hashtags, dollar signs, and many other things to the topic they are interested in to make their tweet reach wider audiences, attracting people with similar tastes. When we try to score the example tweet above using the VADER algorithm and outputting the compound score, we would see that the score would be 0.0, which means that this tweet is entirely neutral with no negative or positive sentiments. We know that this score is wrong, and that can be attributed to the existence of hashtags in the tweet, as this tweet is clearly talking about something positive. Therefore, we need to pre-process the tweets to remove the hashtags, dollar signs, and @ characters from the tweets in this study. Removing the special symbols from the tweet above and then scoring it again would result in the score being 0.4927, which means that the tweet is very positive (indicative of the tweet) and that the investor hopes that the price in the future will increase.

Several tweets have a neutral score even after pre-processing the data; these tweets can further be classified as bot tweets as the words in some bot tweets might not occur in the words to remove in a tweet in our heuristic. Therefore, tweets such as "reply and get 1 Bitcoin" would have a 0.0 compound score, but it is still a tweet made by a bot, which has passed through our heuristic. However, we ignore this score of 0.0 by removing this from the list of sentiments after scoring them the tweets.

In addition, there are specific scenarios that cannot be accounted for when scoring the tweets for a particular token; it is possible that the tweet we are scoring needs to be scored accurately. For example, if there is a tweet, '$ADA is pumping harder than bitcoin'. After pre-processing and scoring it, we will get a score of 0.0, which means that this sentiment analyser fails to analyse the slang in the cryptocurrency topic on Twitter even though this tweet is talking about something positive. This also means that some of the neutral tweets removed in the previous step were false neutrals, meaning they should have had a score, but our sentiment analyser failed to score it accurately because of the terms used within the tweet. Another example where we can say that we cannot account for this possibility is when the tweet for a particular token is scored inaccurately because it is talking positively about some other token or some other thing, an example would be the tweet '$ADA is better than bitcoin because it will lead to higher prices while bitcoin will not' which is retrieved under the query 'bitcoin'. One can see that this tweet is talking about the token Bitcoin in a negative light. Therefore, the Investor Sentiment of this Investor who tweeted this out is unfavourable for Bitcoin and favourable for Cardano. However, our sentiment analyser would give this a positive score of 0.4404, which is not representative of the tweet. But we cannot mitigate this fact with the current algorithm.

If we are to mitigate these effects, then one way might be to look at the distance of the word from our target word (bitcoin in this case) and judge whether the sentence is favourable for that target word. However, that can be used to look into the future to improve this study. Considering everything said above, we hypothesise that the cases in which these anomalies can occur are minimal and, therefore, do not need to be considered in this study and can be an extension for the future works potentially stemming from this study. Doing so will yield us the sentiments of 8 different cryptocurrencies over a period of time which we believe to be more accurately representative of the market sentiment at every given moment in that time.



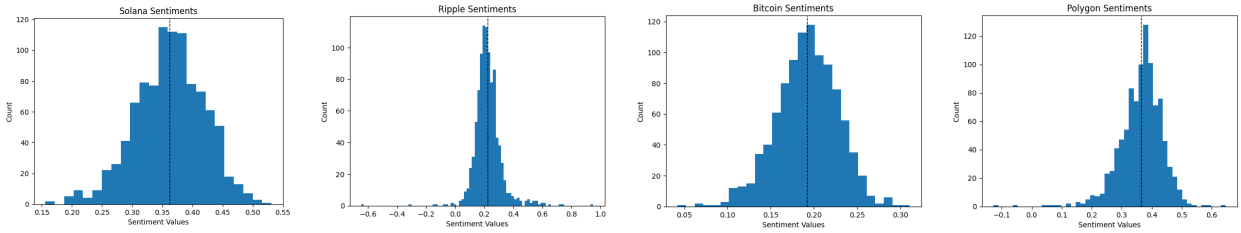
(a) Bitcoin Sentiment **(b)** Polygon Sentiment **(c)** Ripple Sentiment

(d) Polkadot Sentiment **(e)** Dogecoin Sentiment **(f)** Cardano Sentiment

(g) Hedera Sentiment **(h)** Solana Sentiment

Figure 10. Sentiments for different Crypto Tokens

From Figure 10, we can see that the sentiment varies significantly for every crypto token with no observable trend available. Solana and Cardano are the ones that vary significantly. However, the sentiment for both of them remains positive at all times; this might mean that the Investor in the market is very optimistic rather than looking at the actual value of the token. Bitcoin is also showing high variability, but the sentiment is always positive. There is little negativity surrounding Polygon, Polkadot, Dogecoin, and Hedera, with Hedera having the lowest trough. This can be attributed to the noise in the data, as these troughs do not follow a general trend. However, a histogram of the same might give us better insights into what the distribution looks like and if there can be anything said about the distribution.



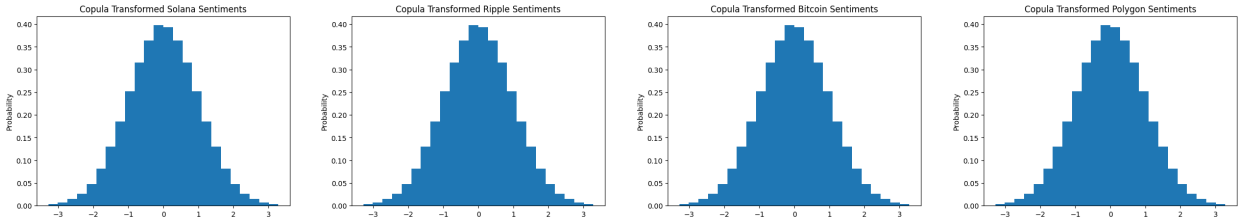
(a) Solana Sentiment **(b)** Ripple Sentiment **(c)** Bitcoin Sentiment **(d)** Polygon Sentiment

Figure 11. Histogram for sentiments for selected Crypto Tokens

Figure 11 shows us how the sentiments are distributed. At first glance at the data, they might seem normally distributed. However, we can see that there cannot be any trend established just by looking at this data, as all of them have significantly different distributions. Another exciting plot to look at would be the distribution after it has been transformed using copulas.

```
import scipy.stats as ss
z_i = ss.norm.ppf((ss.rankdata(DATA_ARRAY, method='average')-0.5)/len((b)))
```

Here, I trasformed the data in the data array for the same crypto tokens shown in Figure 11 and stored them in a variable z_i, and plotted its histogram to see its distribution.



(a) Solana Sentiment **(b)** Ripple Sentiment **(c)** Bitcoin Sentiment **(d)** Polygon Sentiment

Figure 12. Histogram for Copula Transformed Sentiments for selected Crypto Tokens

We can see from Figure 12 that the distribution of all the sentiments is identical, with almost no skewness and kurtosis. We can therefore confirm visually that this data is indeed normal. However we can conduct the JB test onto it to see if it really is normal.

Token	Mean	Standard Deviation	Median	Skew	Kurtosis	JB-Statistic	JB P-Value
Bitcoin (BTC)	0.191	0.035	0.192	-0.265	0.408	18.364	1.02e-04
Polygon (MATIC)	0.363	0.071	0.372	-1.024	4.528	1011.648	2.10e-220
Solana (SOL)	0.362	0.057	0.363	-0.277	0.194	14.188	8.29e-04
Ripple (XRP)	0.225	0.097	0.217	0.464	13.491	7490.182	0.00
Dogecoin (DOGE)	0.287	0.100	0.291	-0.243	2.652	297.915	2.03e-65
Cardano (ADA)	0.328	0.068	0.332	-0.330	0.292	21.357	2.30e-05
Polkadot (DOT)	0.415	0.137	0.432	-0.992	2.485	414.405	1.03e-90
Hedera (HBAR)	0.391	0.150	0.402	-0.984	4.229	891.445	2.66e-194

Table 2. Summary Statistics of Raw Sentiments

Table 2 shows that almost all the raw sentiment distributions for the different crypto tokens are negatively skewed with a positive mean which might mean that an Investor, in general, holds positive sentiment for the token that they are talking about hoping for the current value of the token to catch up to its true value (subjective to the investor). The negative skew might also indicate the presence of fear in the market when the current value of an asset goes down unexpectedly. However, we can say that the Investor Sentiment, in general, is optimistic. This effect is more pronounced in Polygon, Polkadot, and Hedera, as their sentiments are more negatively skewed when compared to the other tokens. Ripple, however, is the only token for which the distribution of the sentiments exhibits a positive skew, even though it is almost symmetrical due to the low skew value. This, in conjunction with a positive mean, might signify that Investor sentiment, in general, is positive for this token. When we combine the above inferences with the kurtosis, we can say that the distributions of Ripple, Polygon, and Hedera are leptokurtic (where the kurtosis is greater than 3). This means that they are thick-tailed, and therefore, there are more chances of outliers present. This can be due to many factors, including extreme sentiments posted by a group of investors (or bots) to drive the prices up, which might result from the market's volatility which resulted in the price going down or up. For Polygon and Hedera, we can say that the outliers would generally be towards the left of the distribution, which means negative sentiments relative to the mean, for ripple it would be the opposite, which exhibits extreme behaviour at its tails (both the tails, but more towards the right side given that the data is almost symmetrical).

Lastly, the tests for normality revealed that the curves are not closely related to normal, which makes it difficult for us to move forward with modelling. Therefore, we would need to transform it using copulas. The JB test after copula transforming the data should confirm to us that the data is indeed normal and, therefore, can be worked on.

Token	JB-Statistic	JB P-Value
Bitcoin (BTC)	0.0323	0.983
Polygon (MATIC)	0.0323	0.983
Solana (SOL)	0.0323	0.983
Ripple (XRP)	0.0323	0.983
Dogecoin (DOGE)	0.0323	0.983
Cardano (ADA)	0.0323	0.983
Polkadot (DOT)	0.0323	0.983
Hedera (HBAR)	0.0586	0.971

Table 3. JB Test after Copula Transformation for Sentiments

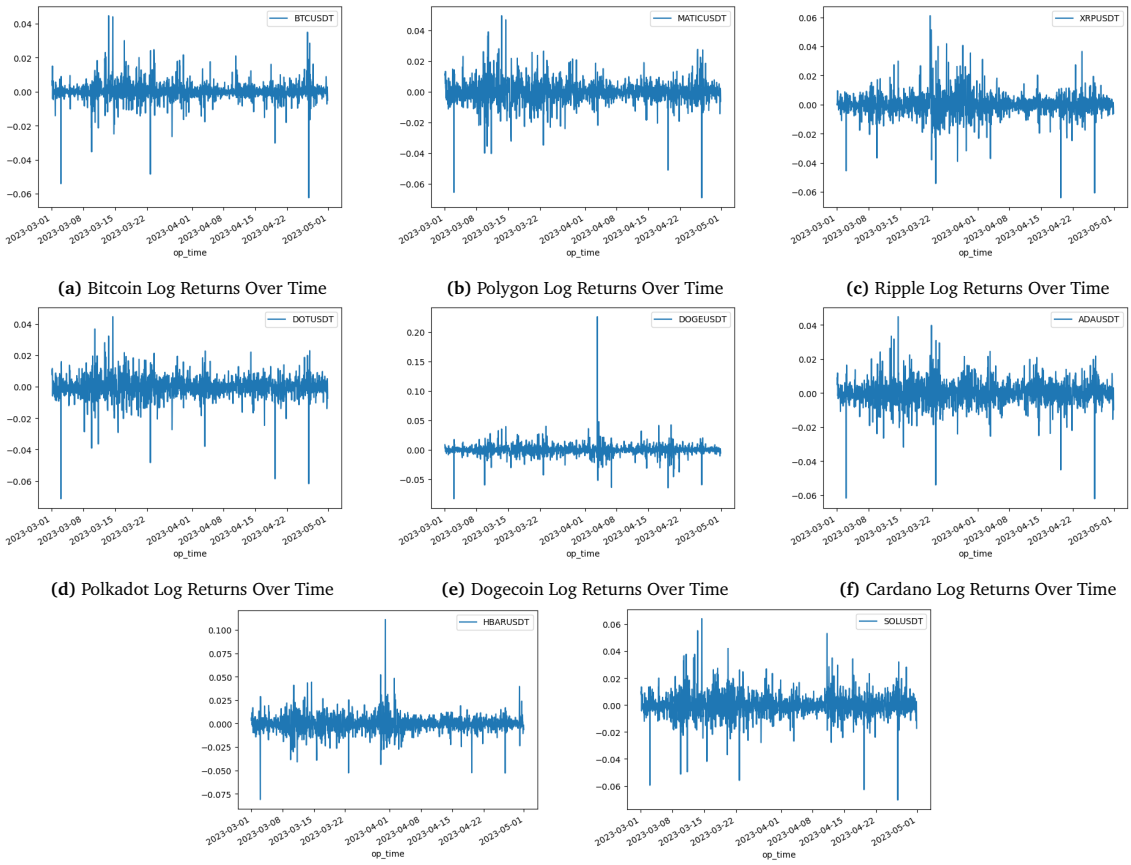
From Table 3, we can confirm that after transformation of the distribution of the sentiments of different tokens, they are distributed normally as given by the low statistic value and high p-value.

5.2 Data Analysis for Prices

The prices of cryptocurrencies do give us some information. However, they are generally not normalised and can be subject to interpretation by comparing and contrasting with other prices in the market. Therefore, we use log returns. It is a central concept in finance and is assumed to be normalised, with the variance giving us the volatility of the asset. Log returns also allow us to analyse the asset scale-free; we do not need to look at potentially exponential growth over time by looking at the growth compared to the previous time scale.

```
pdf = price_dataframe.pct_change().iloc[1:]
```

The code above calculates the percentage change from $t-1$ to t, and then removes the first column because it would give us NaNs, where 'price_dataframe' is the dataframe of the fetched prices that we got in the previous sub-section.



(a) Bitcoin Log Returns Over Time **(b)** Polygon Log Returns Over Time **(c)** Ripple Log Returns Over Time

(d) Polkadot Log Returns Over Time **(e)** Dogecoin Log Returns Over Time **(f)** Cardano Log Returns Over Time

(g) Hedera Log Returns Over Time **(h)** Solana Log Returns Over Time

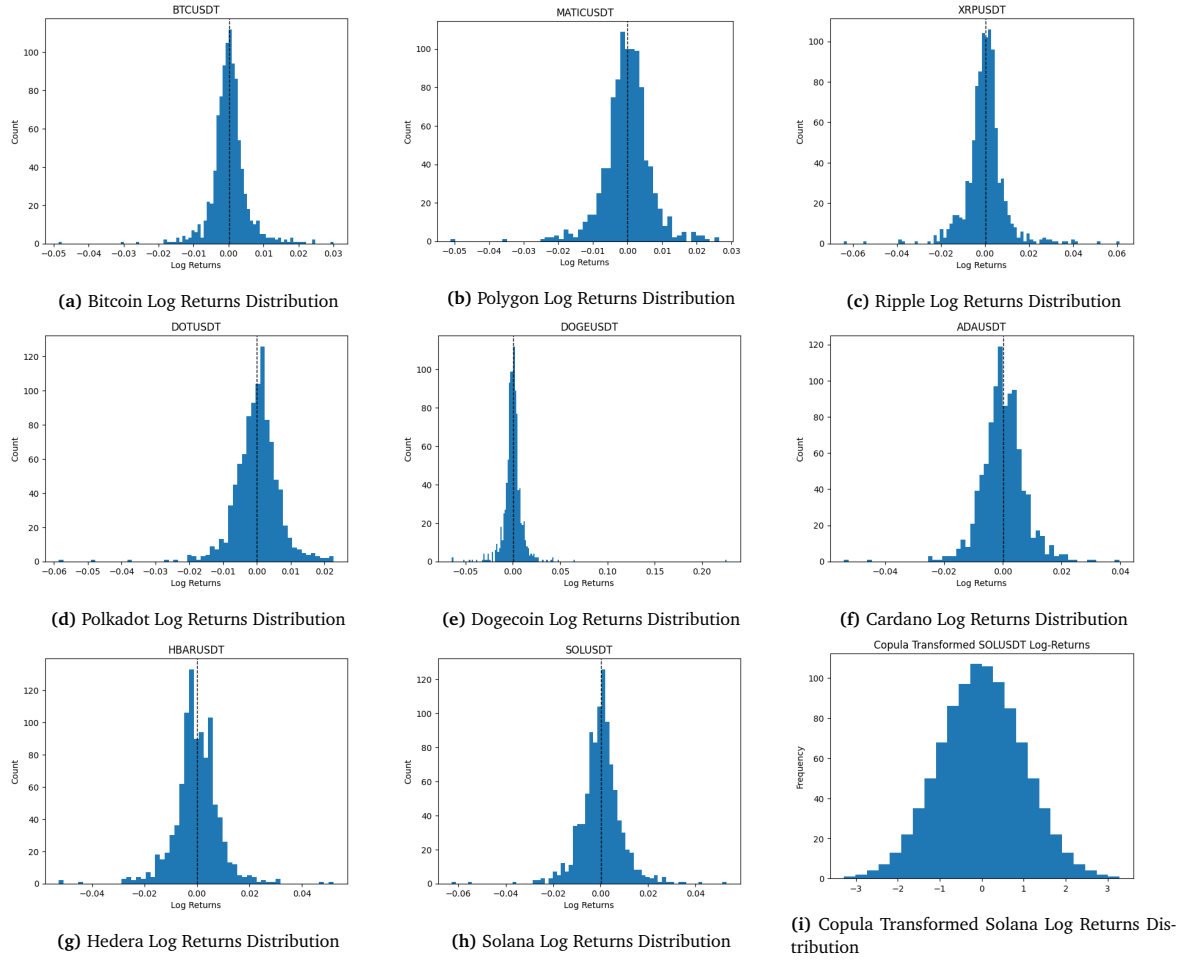
Figure 13. Log Returns Over Time for different Crypto Tokens

We can see in figure 13 that the log-returns of the prices of different tokens shown above are centred around 0.0, which can be confirmed in Table 4. The variance in the log returns of these currencies show the volatility of the asset, which measures the risk of an asset, meaning that the higher the volatility of the asset, the more risky it is to invest in. Looking at Figure 13 in isolation, we can see that Hedera and Bitcoin show less volatility than other tokens, with Hedera almost acting like a stablecoin with minimal returns over time. Dogecoin would have also followed this trend. However, there is a significant anomaly around April 8th for Dogecoin. Solana, Polkadot, Cardano, Polygon, and Ripple show similar return trends, with their variance looking almost the same, with Ripple and Solana showing the highest variance between them. One can also see that when Polygon became a little stable gradually after March 22nd, Ripple started to have high variance. Cardano looks identical to Polygon, and Polkadot has almost the same plot as them both.

When we look at these log returns in conjunction with the sentiment distribution, it is hard to establish any trend. Take, for example, the log return for Ripple after April 1st and the sentiments for Ripple during the same period.

We see that the sentiment variance increases when the returns of tokens stabilise, with some hours having negative sentiments as well. However, the sentiment remained positive when there was relatively high volatility in Ripple. The same can be said about other tokens.

However, as we established earlier, log-returns only give us the impression of being normal, and in reality, they might not be normal for a given financial asset. This can therefore be easily verified by looking at the summary statistics of the distributions created by the log-returns of the crypto token and the histogram of the log-returns. A histogram of the log-returns would give us more insights than the time-series plots that were shown in Figure 13. We will use them both in conjunction with each other to analyse the data.



(a) Bitcoin Log Returns Distribution

(b) Polygon Log Returns Distribution

(c) Ripple Log Returns Distribution

(d) Polkadot Log Returns Distribution

(e) Dogecoin Log Returns Distribution

(f) Cardano Log Returns Distribution

(g) Hedera Log Returns Distribution

(h) Solana Log Returns Distribution

(i) Copula Transformed Solana Log Returns Distribution

Figure 14. Log Returns Distribution for different Crypto Tokens

Figure 14 gives us the distributions of the log-returns of different cryptocurrencies. We can see that these distributions give us the illusion that they are normally distributed. But, when we look at the JB-test statistic and p-value in Table 4, we can clearly establish that these distributions are far from normal.

Token	Mean	Standard Deviation	Median	Skew	Kurtosis	JB-Statistic	JB P-Value
Bitcoin (BTC)	1.44e-04	5.25e-03	0.0	-0.470	12.29	6223.96	0.00
Polygon (MATIC)	-8.57e-05	6.66e-03	0.0	-0.535	5.59	1330.72	1.09e-289
Solana (SOL)	1.82e-04	8.79e-03	0.0	-0.203	6.92	1969.24	0.00
Ripple (XRP)	2.69e-04	8.95e-03	0.0	0.196	9.87	4001.58	0.00
Dogecoin (DOGE)	2.26e-04	1.19e-02	0.0	6.674	133.45	736750.75	0.00
Cardano (ADA)	1.76e-04	7.41e-03	0.0	-0.333	5.697	1347.71	2.22e-293
Polkadot (DOT)	4.53e-05	6.65e-03	0.0	-1.269	10.466	4750.52	0.00
Hedera (HBAR)	-7.90e-05	8.61e-03	0.0	-0.182	6.20	1580.16	0.00

Table 4. Summary Statistics of Log-Returns

We can move on with the analysis of the log-returns by looking at Figure 13, Figure 14, and Table 4, which would be a more complete analysis as before, we only looked at the figures in isolation. We said before that looking at

Hedera and Bitcoin shows less volatility than others. However, that claim, when considered in conjunction with the distribution of the log-returns and the summary statistics, is not justified. One can see that the kurtosis for bitcoin is the second highest in Table 4. This means that the asset is highly volatile. After looking at the distribution, we can make the claim that this is due to the case of a few outliers in the left-tail region, which took place three times between March and May. This does not agree with the general trend of negative skew (albeit almost symmetrical) of the distribution, in which one would expect the outliers to be more in the right tail of the distribution.

Polygon and Cardano exhibit the lowest kurtosis amongst the tokens, which might make them representatives of a safer choice when compared to other tokens, with Cardano being the better choice given that Cardano has less of a skew relative to Polygon and also lesser kurtosis, which is also consistent with the sentiments exhibited by Cardano, which does not change a lot and is not susceptible to outliers due to its low kurtosis.

Hedera and Solana are also good options for making the safe bet, given that the kurtosis and skew of both of them are nearly identical. Hedera is the token with the closest skew value to 0, which means that it is almost symmetrical, more so than every other token, and exhibits a low kurtosis, making it a good choice for a low-risk investment.

Dogecoin is the token that has the highest kurtosis at a staggering 133.45 making this token the most volatile out of all (Notice the anomaly on April 8th in Figure 13) followed by Polkadot at 10.46. Investors would not want to invest in these tokens given their volatility depending on their trading strategies; however, this, when compared to their sentiments, does not seem to suggest that investors are avoiding these tokens, as their distributions are associated with moderate investor sentiments.

The case of Ripple is rather interesting, given the fact that even though Ripple is almost having 0 skew, the high kurtosis might lead the investors to avoid the token, but it might still be preferable to Dogecoin or Polkadot. But, when we take a look at the distribution of the log-returns generated by Ripple in conjunction with its sentiments, then we come to see that the investor sentiments around Ripple varies a lot, which means that the sentiments are quite volatile over time, which might be consistent with the frequency volatility of the prices as opposed to the infrequent volatility of Dogecoin and Polkadot.

Now, to perform advanced time series analysis in our causal framework, we would need to normalise the distributions, which after copula transformations, we can see are normal given the results of the JB test in Table 5. A depiction of the transformed distribution can be found in Figure 14.

Token	JB-Statistic	JB P-Value
Bitcoin (BTC)	3.233e-02	0.983
Polygon (MATIC)	3.231e-02	0.983
Solana (SOL)	3.212e-02	0.984
Ripple (XRP)	3.232e-02	0.983
Dogecoin (DOGE)	3.233e-02	0.983
Cardano (ADA)	3.227e-02	0.984
Polkadot (DOT)	3.229e-02	0.984
Hedera (HBAR)	2.703e-02	0.986

Table 5. JB Test after Copula Transformation for Log-Returns

Just looking at the raw distributions between sentiments and the log-returns of different crypto tokens cannot establish any correlations between them just by looking at them. This might be due to a number of factors, including noise in sentiments which does not produce a good distribution, and less data, amongst other things. Due to this, we would consequently move on to find causal links between them and see if one causes the other to see how these instruments interact from a causal viewpoint and if sentiments have an effect on the returns and vice versa.

6 Application of Causal Contagion to Cryptocurrencies and their Sentiments

The application of the CCM to cryptocurrencies is a relatively straightforward one. After we have the data of the log-returns of the crypto tokens and the sentiments of those tokens, we can form a dataframe that would be the aggregation of all the log returns and sentiments over the same period. This dataframe would then be taken by the NECO function as an argument, and then it will try to find the resulting NECO to model the interaction between the instruments and their sentiments. While finding the NECO, we make a DAG to have a graphical representation of the causal links in our data. At first, these links form a complete graph, but as explained before, we use the PC algorithm to remove the edges and find the directed links between different variables, and we do it at some significance level. This means that to check for independence, and we test a hypothesis at a given alpha value (usually 0.05) to allow maximum true links, normal transformed data allows us to check for said independencies quite easily, which is why

we used copula transformation on our data. At the alpha value of 0.05 and at lag 6, we would have the resulting graph in Figure 15. Here, the log-return node of a token is denoted with a '$' in front of the name of the token, that is, '$BTC' is the log-return on the price of Bitcoin, and 'BTC' is the sentiment of Bitcoin.

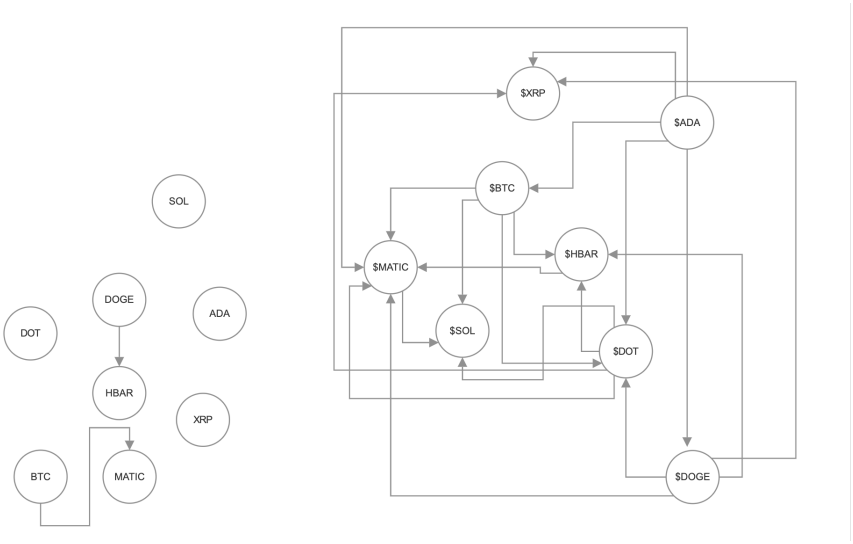


Figure 15. Causal Links at $\alpha = 0.05$

In figure 15, we can see that there is a causal cluster when it comes to the log returns. That means that the log-returns of the instruments impact each other. However, given our data, there is no causal evidence to support the claim that log-returns also impact the sentiments or vice-versa. This was discussed when we were doing the exploratory analysis of the data at hand, and we saw that it was hard to establish any correlation between the prices and sentiments as there was no clear trend available to establish at all. That means that in the current market model of sentiments and log-returns of financial instruments, we do not need to model the dependencies of sentiments as they have no causal impact on the prices. Nevertheless, there is the possibility of a third confounding cluster that would somehow establish the causal dependence between the movement of prices and sentiments. However, this is at a very high significance level. If we increase the alpha bit by bit, we can start unravelling the dependence between the clusters. We would not increase alpha too much as that leaves room for the introduction of false links, which would not give the desired results and not produce a model representative of the market. Therefore, by just increasing alpha to about 0.4, we would see the first instances of dependence between log-returns and sentiments. The graph of the perturbation can be seen in Figure 16.

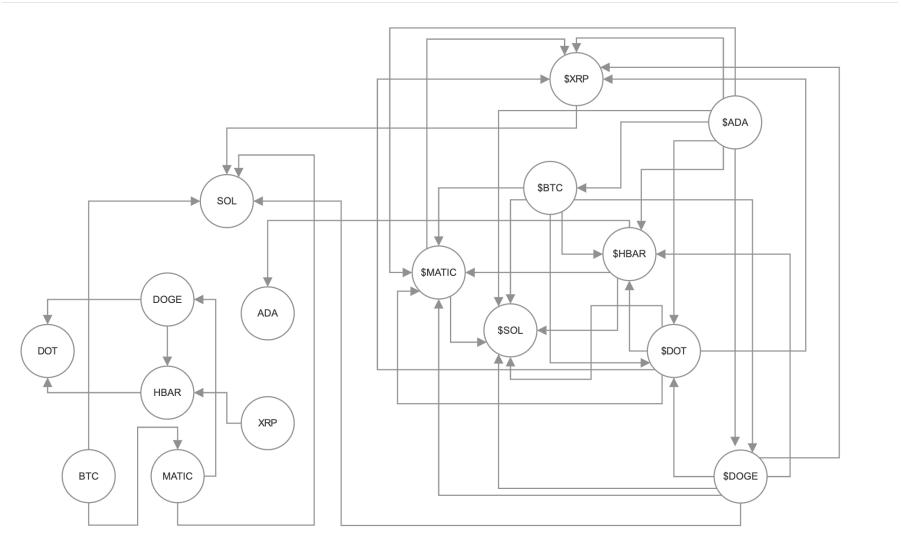


Figure 16. Causal Links at $\alpha = 0.4$

In figure 16, we can see the same cluster of prices. However, links are going out to sentiments from prices. That means that even at a high alpha value, we cannot see if the sentiments are causing the prices to move. However, we

see that the prices affect the sentiments, which might be consistent with the market given that people react to prices; therefore, the result is not all that surprising. This is also consistent with what we found at a lower significance level, whereby we would not need to model the sentiments to model the market landscape potentially.

The causal Contagion Framework uses the PC algorithm inherently, which we have touched upon earlier, and the PC algorithm inherently is order dependent. That means it might produce a different DAG for a different permutation of the input nodes as it might encounter some conflicts while trying to orient the links. In order to counteract that, we can force the algorithm to produce a fully order-independent graph by following the 'conservative' settings of the PC algorithm [9], which might yield a CPDAG, a valid output from the PC algorithm that can be attributed to multiple factors. Some of these factors might be the presence of excess noise in the distributions (in sentiments, it can be attributed to false positives, which makes causal discovery very difficult), having fewer data points also plays a part, or not having enough nodes in the network might also be a potential source of the problem. Therefore, to form a DAG from the resultant CPDAG, one might use their expertise in the field they are modelling and give an educated order to the input variables. Alternatively, one could change the potential sources of error highlighted above.

7 Conclusion and Discussion

We set out to discover how prices and sentiments interact in the cryptocurrency market. To do so, we used a causal framework whose application is still very nascent in the field of financial market modelling. While the other financial markets have been around for a lot of years, the cryptocurrency market is also relatively new, and therefore there is not much research into how the different tokens interact.

We found no significant causal evidence of any link between prices and sentiments. Therefore, we can conclude that prices of the currencies move independently from the investor sentiments on Twitter, in contrast with some studies that show a bi-directional correlational relationship between sentiments and prices. However, suppose we work at a lower significance level at $\alpha = 0.4$. In that case, we can see that the prices of the tokens impact the investor sentiment on Twitter, which might represent the reality whereby the investor on Twitter is just reacting to the happenings in the market rather than the market reacting to the sentiments of investors on Twitter.

A direct extension of this study could be to predict the prices using the causal model of the market. This model will be more robust than a model based on correlation networks. Instead, it is based on causal networks. By extension, this model will perform better because it encapsulates the underlying causal model of the market and is therefore not susceptible to spurious correlations. Another extension might be to consider that the scores of the sentiments and the separation of the tweets is not optimal, and an improvement can be made on the VADER algorithm so that we accurately measure the sentiments for a given token. Increasing the sample size of the tweets and prices, and also the nodes would be the most apparent extension and might lead to different causal structures as there will be more nodes and more data in the graph, which would result in a representative model of the market.

References

[1] A. Beattie. The pioneers of technical analysis. *URL: https://www.investopedia.com/articles/financial-theory/10/pioneers-technical-analysis.asp*, 2021.

[2] J. Bollen, H. Mao, and X. Zeng. Twitter mood predicts the stock market. *Journal of computational science*, 2(1):1–8, 2011.

[3] E. Bouri, P. Molnár, G. Azzi, D. Roubaud, and L. I. Hagfors. On the hedge and safe haven properties of bitcoin: Is it really more than a diversifier? *Finance Research Letters*, 20:192–198, 2017.

[4] G. W. Brown and M. T. Cliff. Investor sentiment and the near-term stock market. *Journal of empirical finance*, 11(1):1–27, 2004.

[5] D. M. Chickering. Learning equivalence classes of bayesian-network structures. *The Journal of Machine Learning Research*, 2:445–498, 2002.

[6] P. Ciaian, M. Rajcaniova, et al. Virtual relationships: Short-and long-run evidence from bitcoin and altcoin markets. *Journal of International Financial Markets, Institutions and Money*, 52:173–195, 2018.

[7] P. Ciaian, M. Rajcaniova, and d. Kancs. The economics of bitcoin price formation. *Applied economics*, 48(19):1799–1815, 2016.

[8] S. Colianni, S. Rosales, and M. Signorotti. Algorithmic trading of cryptocurrency based on twitter sentiment analysis. *CS229 Project*, 1(5):1–4, 2015.

[9] D. Colombo, M. H. Maathuis, et al. Order-independent constraint-based causal structure learning. *J. Mach. Learn. Res.*, 15(1):3741–3782, 2014.

[10] F. Dablander. An introduction to causal inference. 2020.

[11] T. Developer. Twitter rate limit. `https://developer.twitter.com/en/docs/rate-limits`. Accessed: 2023-05-05.

[12] Erwin. Do professional traders use technical analysis? *URL: https://improve-your.trade/blog/do-professional-traders-use-technical-analysis*.

[13] E. F. Fama. Efficient capital markets: A review of theory and empirical work. *The journal of Finance*, 25(2):383–417, 1970.

[14] D. Geiger, T. Verma, and J. Pearl. Identifying independence in bayesian networks. *Networks*, 20(5):507–534, 1990.

[15] C. Glymour, K. Zhang, and P. Spirtes. Review of causal discovery methods based on graphical models. *Frontiers in genetics*, 10:524, 2019.

[16] A. Hayes. Technical analysis: What it is and how to use it in investing. *URL: https://www.investopedia.com/terms/t/technicalanalysis.asp*, 2022.

[17] C. Hicks. Different types of cryptocurrencies. *URL: https://www.forbes.com/advisor/investing/cryptocurrency/different-types-of-cryptocurrencies*, 2023.

[18] D. Hume and E. Steinberg. *An Enquiry Concerning Human Understanding;[with] A letter from a gentleman to his friend in Edinburgh;[and] An abstract of a Treatise of human nature*. Hackett Publishing, 1993.

[19] C. Hutto and E. Gilbert. Vader: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the international AAAI conference on web and social media*, volume 8, pages 216–225, 2014.

[20] C. M. Jarque and A. K. Bera. A test for normality of observations and regression residuals. *International Statistical Review/Revue Internationale de Statistique*, pages 163–172, 1987.

[21] M. I. Jordan. Graphical models. 2004.

[22] O. Kraaijeveld and J. De Smedt. The predictive power of public twitter sentiment for forecasting cryptocurrency prices. *Journal of International Financial Markets, Institutions and Money*, 65:101188, 2020.

[23] L. Kristoufek. Bitcoin meets google trends and wikipedia: Quantifying the relationship between phenomena of the internet era. *Scientific reports*, 3(1):3415, 2013.

[24] X. Li and C. A. Wang. The technology and economic determinants of cryptocurrency exchange rates: The case of bitcoin. *Decision support systems*, 95:49–60, 2017.

[25] A. W. Lo. The adaptive markets hypothesis: Market efficiency from an evolutionary perspective. *Journal of Portfolio Management, Forthcoming*, 2004.

[26] A. W. Lo. Adaptive markets and the new world order (corrected may 2012). *Financial Analysts Journal*, 68(2):18–29, 2012.

[27] C. Meek. Causal inference and causal explanation with background knowledge. *arXiv preprint arXiv:1302.4972*, 2013.

[28] S. Nakamoto. Bitcoin whitepaper. *URL: https://bitcoin.org/bitcoin.pdf*, 2008.

[29] B. Neal. Introduction to causal inference. 2015.

[30] J. Pearl. *Probabilistic reasoning in intelligent systems: networks of plausible inference*. Morgan kaufmann, 1988.

[31] J. Pearl. Causal inference in statistics: An overview. 2009.

[32] J. Pearl. The do-calculus revisited. *arXiv preprint arXiv:1210.4852*, 2012.

[33] J. Pearl. On the interpretation of do (x). *Journal of Causal Inference*, 7(1), 2019.

[34] J. Pearl. The seven tools of causal inference, with reflections on machine learning. *Communications of the ACM*, 62(3):54–60, 2019.

[35] M. Pericoli and M. Sbracia. A primer on financial contagion. *Journal of economic surveys*, 17(4):571–608, 2003.

[36] K. Peters. Binance exchange. *https://www.investopedia.com/terms/b/binance-exchange.asp,*, 2023.

[37] R. L. Peterson. *Trading on sentiment: The power of minds over markets*. John Wiley & Sons, 2016.

[38] L. Quigley and D. Ramsey. Statistical analysis of the log returns of financial assets. *BSc in Financial Mathematics. University of Muenster, Germany*, 2008.

[39] H. Reichenbach. *The direction of time*, volume 65. Univ of California Press, 1956.

[40] K. Rigana, E. C. Wit, and S. Cook. Effects of financial crises on contagion in currency markets. *Available at SSRN 3990153*, 2021.

[41] D. Sayce. Number of Tweets Per Second. https://www.dsayce.com/social-media/tweets-day/. Accessed: 2023-05-05.

[42] R. Sharma. Bitcoin and cryptocurrency's 2020 turnaround. *URL: https://www.investopedia.com/bitcoin-and-cryptocurrencys-2020-turnaround-5094080*, 2021.

[43] P. Spirtes and C. Glymour. An algorithm for fast recovery of sparse causal graphs. *Social science computer review*, 9(1):62–72, 1991.

[44] P. Spirtes and K. Zhang. Causal discovery and inference: concepts and recent methodological advances. In *Applied informatics*, volume 3, pages 1–28. SpringerOpen, 2016.

[45] T. Thadewald and H. Büning. Jarque–bera test and its competitors for testing normality–a power comparison. *Journal of applied statistics*, 34(1):87–105, 2007.

[46] R. S. Tsay. *Analysis of financial time series*. John wiley & sons, 2005.

[47] C. Van Rijckeghem and B. Weder. Sources of contagion: is it finance or trade? *Journal of international Economics*, 54(2):293–308, 2001.

[48] E. W. Weisstein. Normal distribution. *https://mathworld. wolfram. com/*, 2002.

[49] S. Wright. Correlation and causation. 1921.

[50] D. Yermack. Is bitcoin a real currency? an economic appraisal. In *Handbook of digital currency*, pages 31–43. Elsevier, 2015.